

Discussion Unit: DSE Analytics, Persistence

- Understand the role of persistence when using DSE Analytics

Discussion Lab:

Discuss-

DSE Analytics; Optimization (thus far)

DSE Analytics: Optimization (thus far) ..



Source: <https://www.amazon.com/gp/product/0060929510>

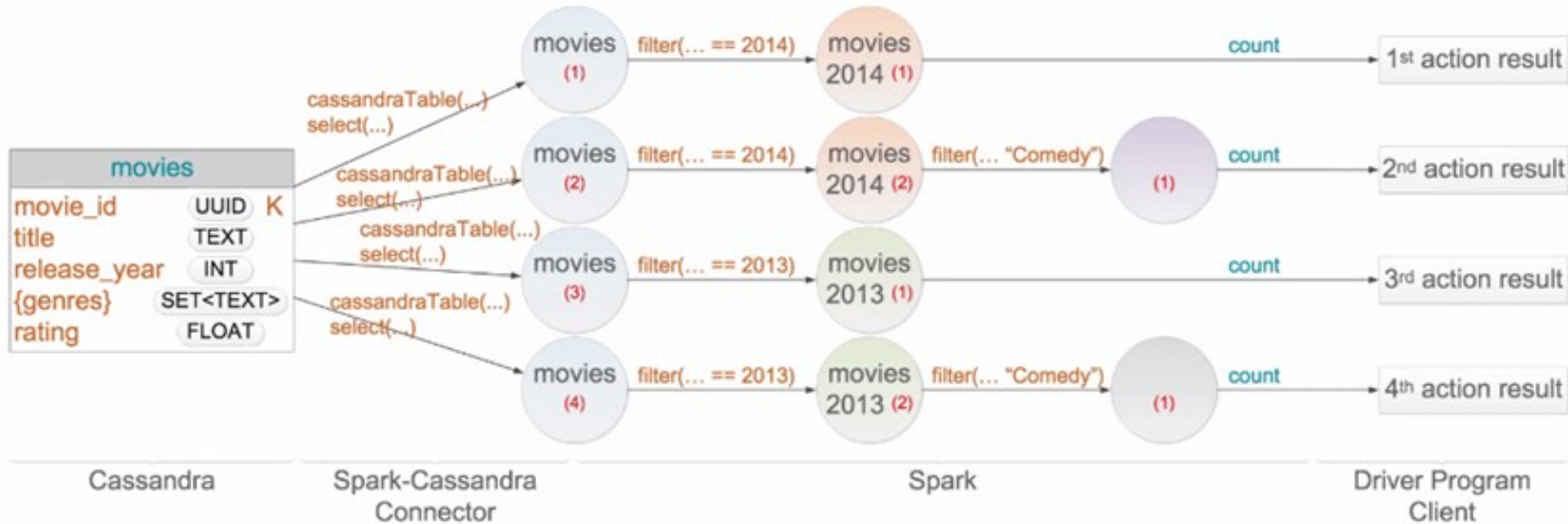
**Based on what you know
right now; How can you
make your Spark Job
faster, more efficient ?**



End of Discussion Lab:

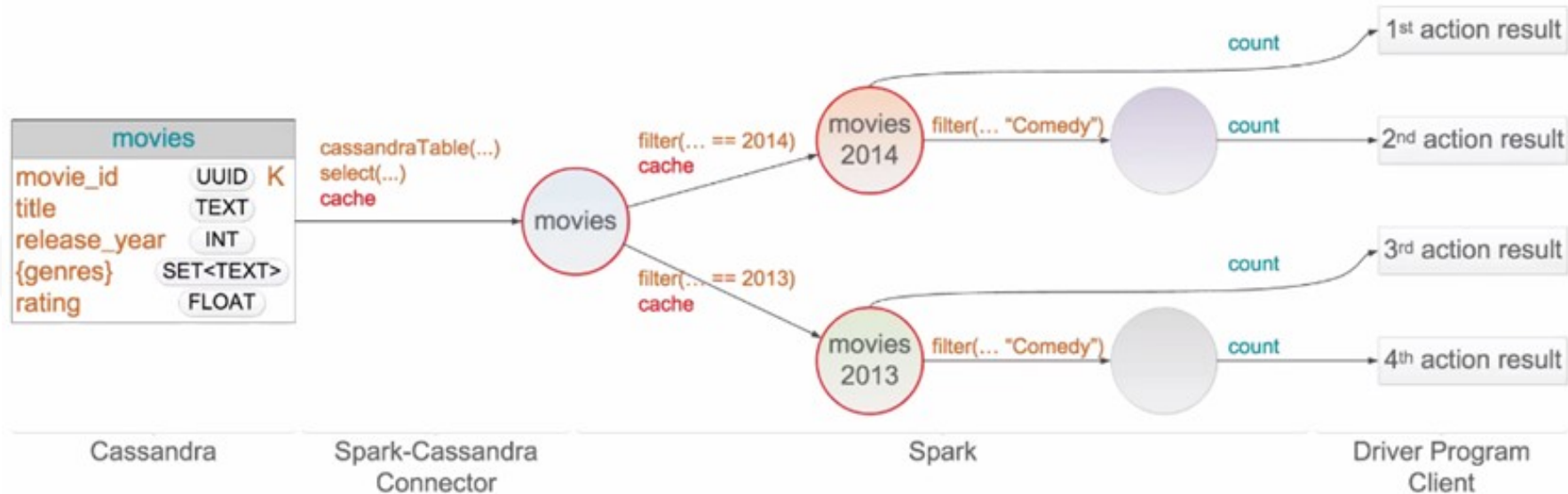
Why we persist- (And recovery from failure)

Image From: DSA-320



Why we persist- (And recovery from failure)

Image From: DSA-320



RDD/DataFrame/DataSet: Persistence API-

Transform	Description
persist(sl)	Persists source RDD According to storage level (sl) specified. Default is MEMORY_ONLY, which persists RDD element as deserialized Java objects in the JVM.
cache()	Same as persist(StorageLevel.MEMORY_ONLY)
unpersist()	Optional; Spark automatically manages this.

Storage Levels

Storage Level	Description
MEMORY_ONLY MEMORY_ONLY_SER	Deserialized, or Serialized, Java objects in the JVM. Partitions that do not fit are not stored, and are recalculated
MEMORY_AND_DISK MEMORY_AND_DISK_SER	Same as above; partitions not fitting in memory are stored to disk
DISK_ONLY	Persisting to disk only.
MEMORY_ONLY_2 MEMORY_AND_DISK_2	Same as above; replication to two nodes
OFF_HEAP	Serialized in-memory using Tachyon; a memory-centric distributed storage system

** Serialized: more space efficient, more CPU intensive


```
val c_rows = sc.cassandraTable("ks_7571", "customer").  
  select("customer_num", "state").  
  as((customer_num:Int, state:String) =>  
    (customer_num, (state)))
```



```
val o_rows = sc.cassandraTable("ks_7571", "orders").  
  select("customer_num", "order_num", "paid_date").  
  as((customer_num:Int, order_num:Int,  
    paid_date:String) => (customer_num,  
      (order_num, paid_date)))
```



```
val j_rows1 = o_rows.join(c_rows)
```



```
val j_rows2 = o_rows.leftOuterJoin(c_rows).  
  filter{case (pk, (row_o, row_c)) => row_c.isDefined}
```



```
val j_rows3 = o_rows.leftOuterJoin(c_rows).  
  filter{case (pk, (row_o, row_c)) => row_c.isDefined}.  
  map{r => ( r._1, Option(r._2._1._2), r._2._1._1, r._2._2 )}.  
  map{r => ( r._1, r._2.getOrElse("N/A"), r._3, r._4.get )}
```



```
j_rows4.saveToCassandra("ks_7571", "customer_orders",  
  SomeColumns("customer_num", "paid_date",  
    "order_num", "state"))
```



Example: When to apply

- Persist ? What level ?



Outer, and isDefined ?



End of Unit: